

Profundización

José Armando Ramírez López

Sofia Montero Castiblanco

Fundación Universitaria Compensar

facultad de ingeniería, ingeniería de software

septiembre 2025

Introducción

El presente informe describe el desarrollo de una aplicación móvil creada con **Ionic + React + Capacitor**, la cual implementa **autenticación JWT** y una **galería de fotos** que permite al usuario capturar imágenes desde la cámara del dispositivo (emulador Android en este caso) y almacenarlas localmente.

Este proyecto se realiza como parte de la metodología de aprendizaje basado en proyectos (ABP), integrando conceptos de **seguridad (JWT)**, **uso de plugins de Capacitor**, y **migración de aplicaciones web a plataformas móviles**.

Objetivos

Objetivo general

Desarrollar una aplicación híbrida con Ionic/React que integre autenticación mediante **JWT** y un servicio de **galería de fotos** usando la cámara del dispositivo.

Objetivos específicos

- Implementar un sistema de **login** con almacenamiento seguro del token JWT.
 - Configurar **Axios con interceptor** para el consumo de servicios API protegidos.
 - Implementar un **servicio de cámara** y almacenamiento local con Capacitor.
 - Migrar el proyecto web a **plataforma Android** mediante Capacitor y Android Studio.
 - Documentar el proceso de implementación con evidencias gráficas.
-

Desarrollo del proyecto

Creación y configuración del proyecto

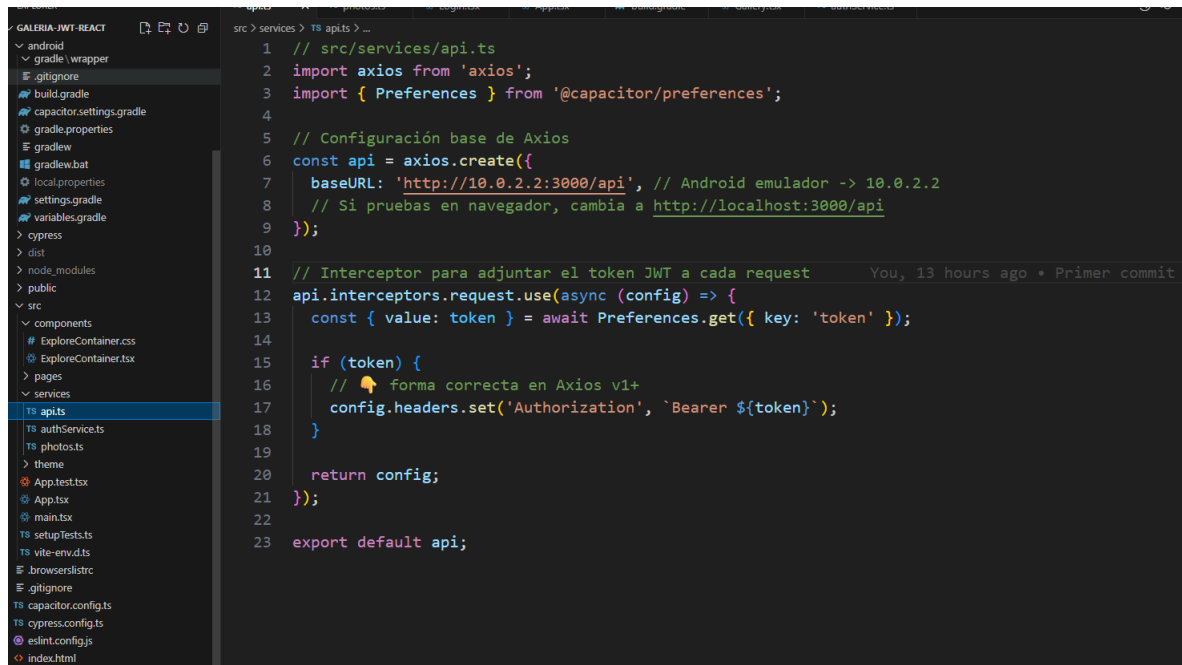
Se creó el proyecto en Ionic con React, configurando dependencias y Capacitor:

```
C:\xampp\htdocs>ionic start galeria-jwt-react blank --type=react
✓ Preparing directory .\galeria-jwt-react in 5.34ms
✓ Downloading and extracting blank starter in 3.21s
> ionic integrations enable capacitor --quiet -- galeria-jwt-react io.ionic.starter
> npm.cmd i --save -E @capacitor/core@latest
npm warn deprecated abab@2.0.6: Use your platform's native atob() and btoa() methods instead
npm warn deprecated domexception@4.0.0: Use your platform's native DOMException instead

added 710 packages, and audited 711 packages in 7m
```

3.2 Configuración de la API y JWT

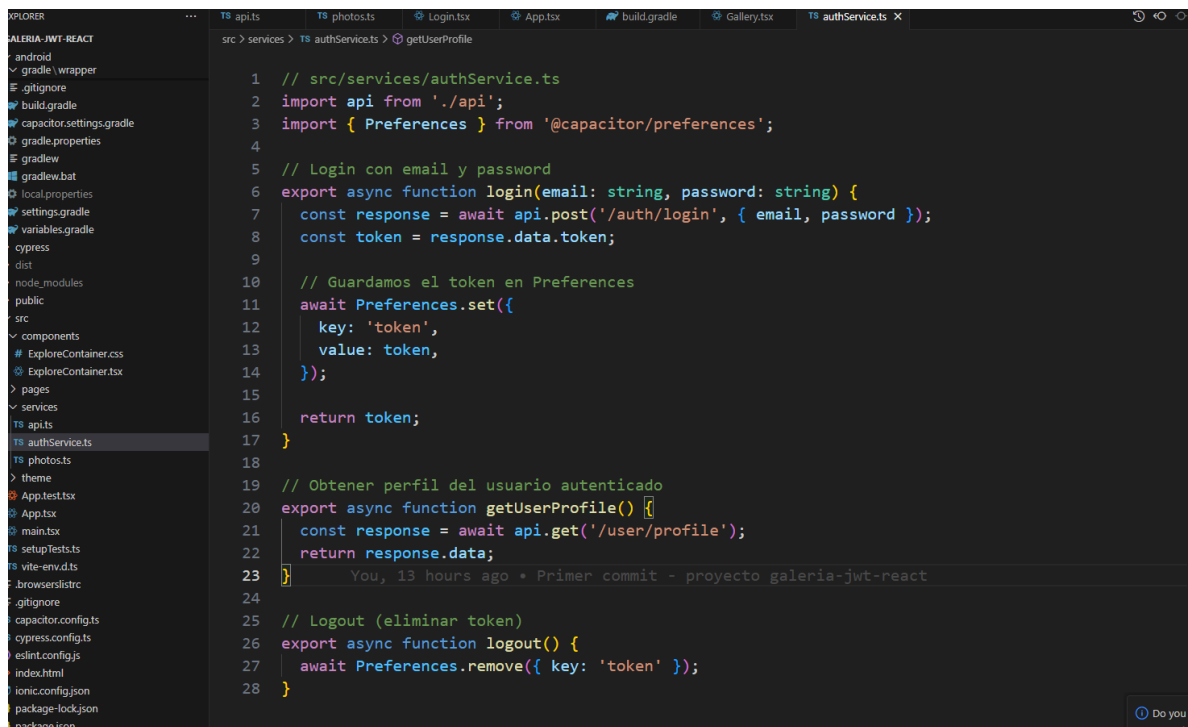
Se creó el archivo `src/services/api.ts` con la configuración base de Axios y un interceptor que añade automáticamente el token JWT a cada petición.



```
1 // src/services/api.ts
2 import axios from 'axios';
3 import { Preferences } from '@capacitor/preferences';
4
5 // Configuración base de Axios
6 const api = axios.create({
7   baseURL: 'http://10.0.2.2:3000/api', // Android emulador -> 10.0.2.2
8   // Si pruebas en navegador, cambia a http://localhost:3000/api
9 });
10
11 // Interceptor para adjuntar el token JWT a cada request
12 api.interceptors.request.use(async (config) => {
13   const { value: token } = await Preferences.get({ key: 'token' });
14
15   if (token) {
16     // 📌 forma correcta en Axios v1+
17     config.headers.set('Authorization', `Bearer ${token}`);
18   }
19
20   return config;
21 });
22
23 export default api;
```

3.3 Servicio de autenticación

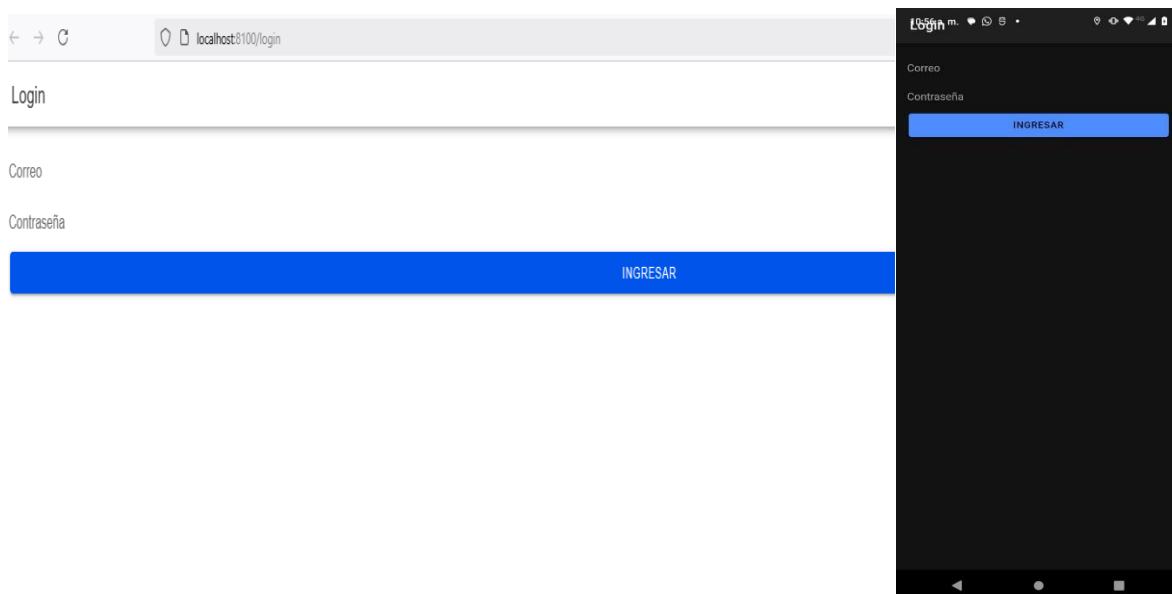
En `src/services/authService.ts` se implementaron las funciones de **login**, **obtener perfil**, y **logout**, utilizando `@capacitor/preferences` para almacenar de forma segura el token.



```
1 // src/services/authService.ts
2 import api from '../api';
3 import { Preferences } from '@capacitor/preferences';
4
5 // Login con email y password
6 export async function login(email: string, password: string) {
7   const response = await api.post('/auth/login', { email, password });
8   const token = response.data.token;
9
10  // Guardamos el token en Preferences
11  await Preferences.set({
12    key: 'token',
13    value: token,
14  });
15
16  return token;
17 }
18
19 // Obtener perfil del usuario autenticado
20 export async function getUserProfile() {
21   const response = await api.get('/user/profile');
22   return response.data;
23 }
24
25 // Logout (eliminar token)
26 export async function logout() {
27   await Preferences.remove({ key: 'token' });
28 }
```

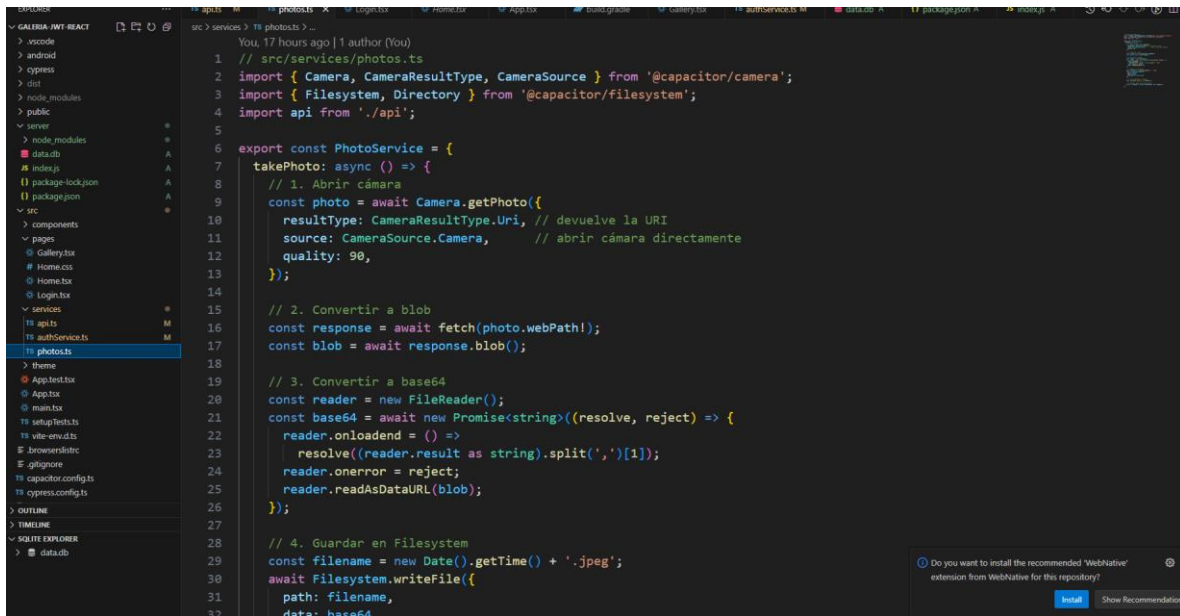
3.4 Página de Login

Se creó la vista de login en `src/pages/Login.tsx`, donde el usuario ingresa credenciales y se guarda el token JWT tras la validación en la API.



3.5 Servicio de fotos

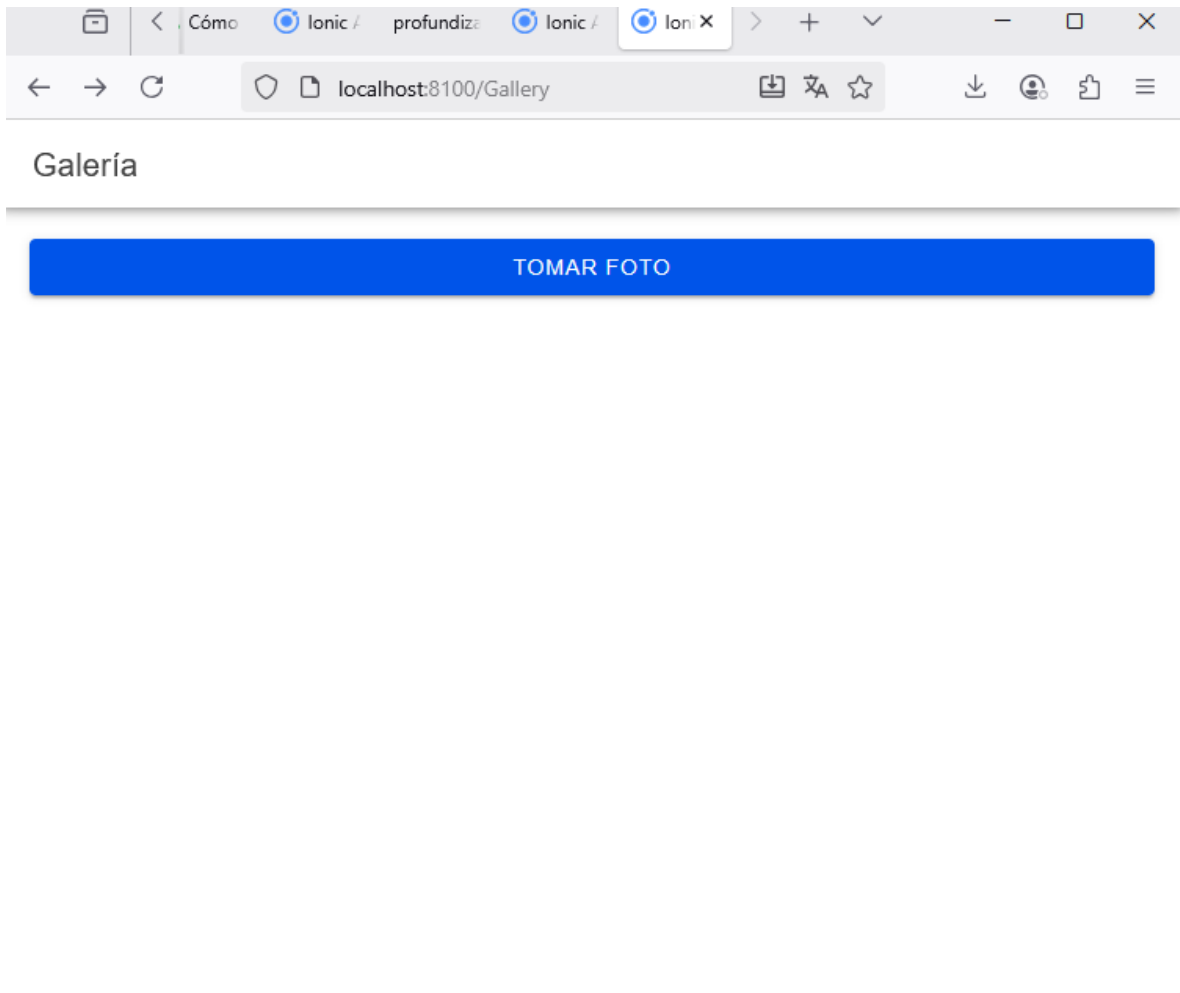
En `src/services/photos.ts` se implementó el uso de la cámara mediante el plugin de Capacitor. Cada foto se convierte a base64 y se guarda en el sistema de archivos del dispositivo.



```
1 // src/services/photos.ts
2 import { Camera, CameraResultType, CameraSource } from '@capacitor/camera';
3 import { Filesystem, Directory } from '@capacitor/filesystem';
4 import api from './api';
5
6 export const PhotoService = {
7   takePhoto: async () => {
8     // 1. Abrir cámara
9     const photo = await Camera.getPhoto({
10       resultType: CameraResultType.Uri, // devuelve la URI
11       source: CameraSource.Camera, // abrir cámara directamente
12       quality: 90,
13     });
14
15     // 2. Convertir a blob
16     const response = await fetch(photo.webPath!);
17     const blob = await response.blob();
18
19     // 3. Convertir a base64
20     const reader = new FileReader();
21     const base64 = await new Promise<string>((resolve, reject) => {
22       reader.onloadend = () => {
23         resolve((reader.result as string).split(',')[1]);
24       };
25       reader.onerror = reject;
26       reader.readAsDataURL(blob);
27     });
28
29     // 4. Guardar en Filesystem
30     const filename = new Date().getTime() + '.jpeg';
31     await Filesystem.writeFile({
32       path: filename,
33       data: base64,
```

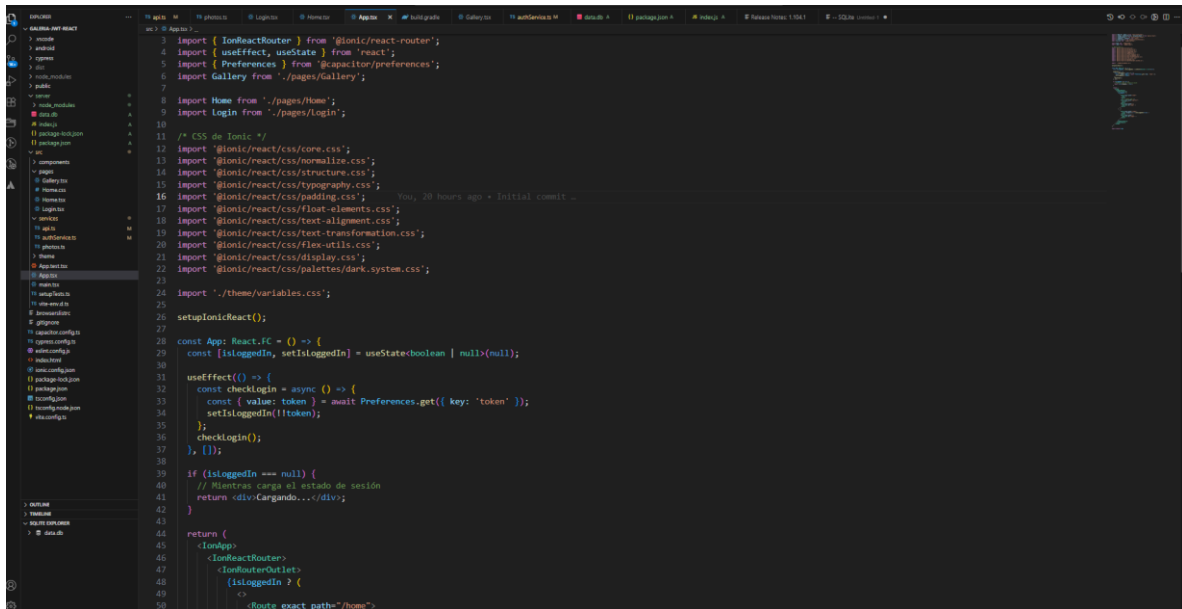
3.6 Página de galería

En `src/pages/Gallery.tsx` se implementó la UI que permite capturar fotos y listarlas en una galería.



Integración en la App

En App.tsx se configuró la lógica de rutas, validando el estado de autenticación y mostrando **Login** o **Home/Gallery** según corresponda.



```
import { IonReactRouter } from '@ionic/react-router';
import { useEffect, useState } from 'react';
import { Preferences } from '@capacitor/preferences';
import Gallery from './pages/Gallery';
import Home from './pages/Home';
import Login from './pages/Login';

/* CSS de Ionic */
import '@ionic/react/css/core.css';
import '@ionic/react/css/normalize.css';
import '@ionic/react/css/structure.css';
import '@ionic/react/css/typography.css';
import '@ionic/react/css/padding.css';
import '@ionic/react/css/floating-elements.css';
import '@ionic/react/css/text-alignment.css';
import '@ionic/react/css/text-transformation.css';
import '@ionic/react/css/flex-utils.css';
import '@ionic/react/css/display.css';
import '@ionic/react/css/palettes/dark.system.css';

import './theme/variables.css';

setupIonicReact();

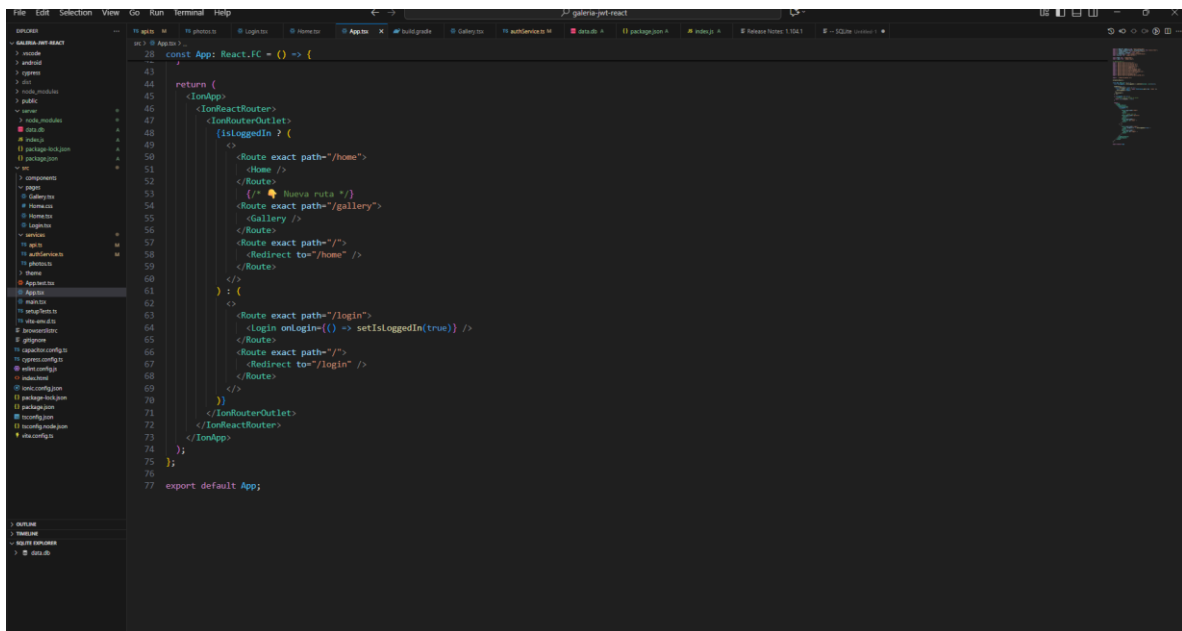
const App: React.FC = () => {
  const [isLoggedIn, setIsLoggedIn] = useState<boolean> | null>(null);

  useEffect(() => {
    const checkLogin = async () => {
      const { value: token } = await Preferences.get({ key: 'token' });
      setIsLoggedIn(!!token);
      checkLogin();
    }, []);

    if (isLoggedIn === null) {
      // Mientras carga el estado de sesión
      return <div>Cargando...</div>;
    }

    return (
      <IonApp>
        <IonReactRouter>
          <IonRouterOutlet>
            {isLoggedIn ? (
              <Route exact path="/home">

```



```

              <Route exact path="/home">
                <Home />
              </Route>
              <Route exact path="/gallery">
                <Gallery />
              </Route>
              <Route exact path="/">
                <Redirect to="/home" />
              </Route>
            ) : (
              <Route exact path="/login">
                <Login onLogin={() => setIsLoggedIn(true)} />
              </Route>
              <Route exact path="/">
                <Redirect to="/login" />
              </Route>
            )
          </IonRouterOutlet>
        </IonReactRouter>
      </IonApp>
    );
  };

  export default App;
}
```

Migración a Android

Se ejecutaron los comandos para migrar la aplicación a Android:

```
C:\xampp\htdocs\galeria-jwt-react>npm install @capacitor/android
```

```
added 1 package, and audited 799 packages in 5s
```

```
187 packages are looking for funding
  run 'npm fund' for details
```

```
C:\xampp\htdocs\galeria-jwt-react>npx cap add android
```

```
✓ Adding native android project in android in 338.77ms
```

```
✓ add in 339.65ms
```

```
✓ Copying web assets from dist to android\app\src\main\assets\public in 38.69ms
```

```
✓ Creating capacitor.config.json in android\app\src\main\assets in 29.30ms
```

```
✓ copy android in 140.55ms
```

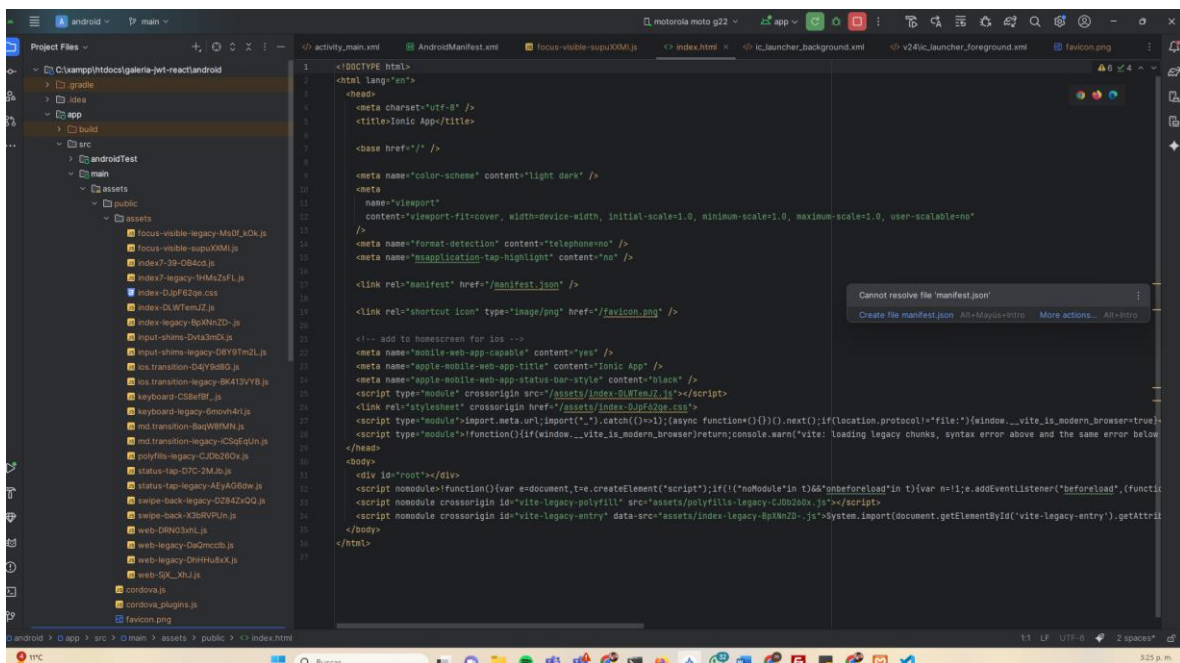
```
C:\xampp\htdocs\galeria-jwt-react>npx cap sync
```

```
✓ Copying web assets from dist to android\app\src\main\assets\public in 63.
```

```
✓ Creating capacitor.config.json in android\app\src\main\assets in 5.97ms
```

```
✓ copy android in 136.46ms
```

Proyecto en AndroidStudio



Resultados

El resultado es una aplicación híbrida funcional con las siguientes características:

- Sistema de login con **JWT**.
- Acceso a galería protegida tras autenticación.

- Captura de fotos desde cámara en emulador Android.
 - Visualización de las imágenes tomadas en la aplicación.
 - Proyecto migrado y probado en Android Studio.
-

Conclusiones

- La integración de **JWT** en aplicaciones móviles híbridas permite un acceso seguro a servicios API REST.
 - El uso de **Capacitor** facilita el acceso a funcionalidades nativas como la cámara y el almacenamiento local.
 - Ionic con React brinda un entorno de desarrollo rápido y portable para aplicaciones móviles.
 - La migración a Android con Capacitor permite ejecutar en dispositivos móviles de forma nativa.
-

Referencias

- CapacitorJS. (2025). *Capacitor Documentation*. Recuperado de: <https://capacitorjs.com/docs>
- Ionic Framework. (2025). *Ionic React Documentation*. Recuperado de: <https://ionicframework.com/docs/react>
- Axios. (2025). *Axios GitHub Repository*. Recuperado de: <https://github.com/axios/axios>
- JWT.io. (2025). *Introduction to JSON Web Tokens*. Recuperado de: <https://jwt.io/introduction/>